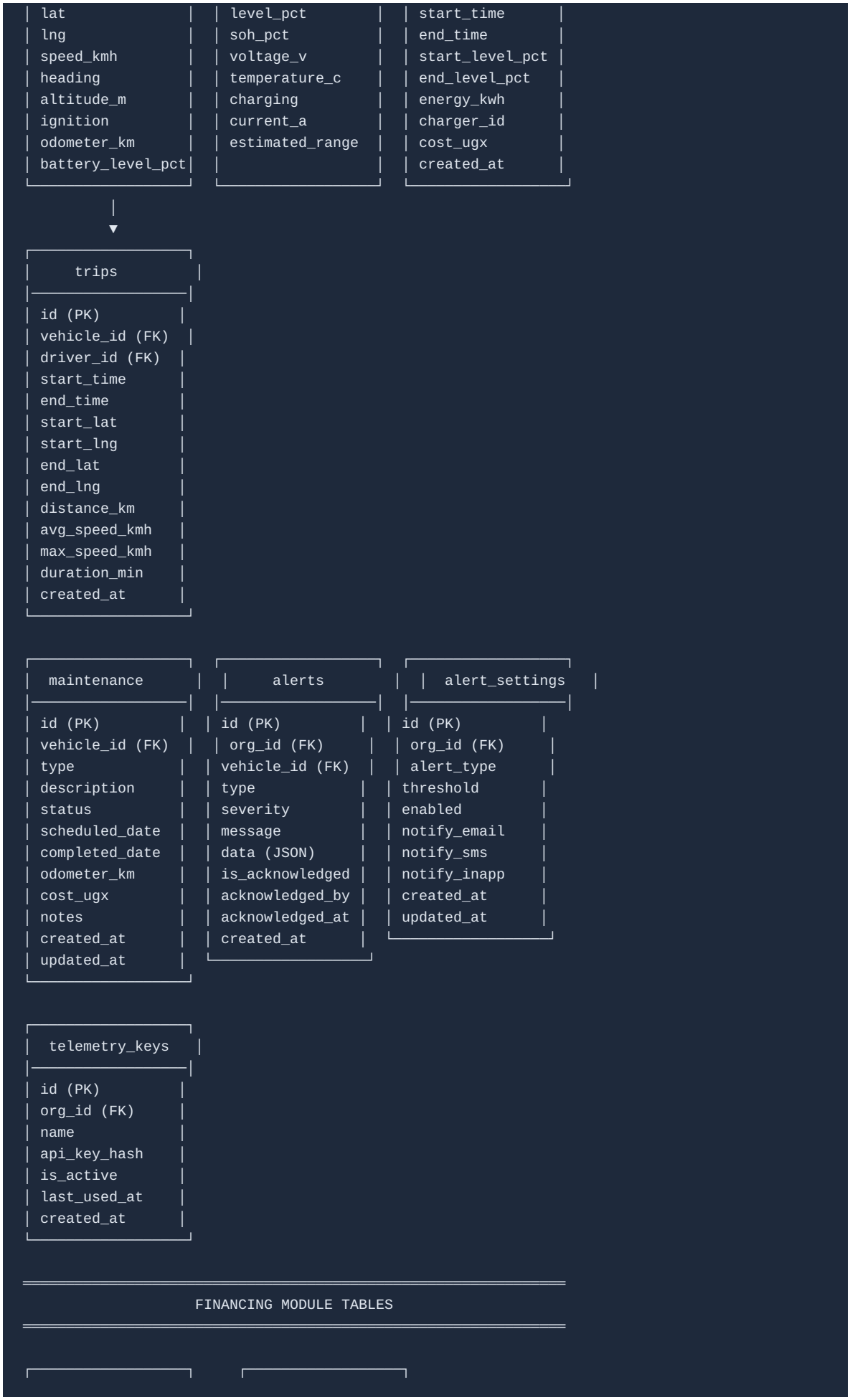
FleetSpark — Database Schema

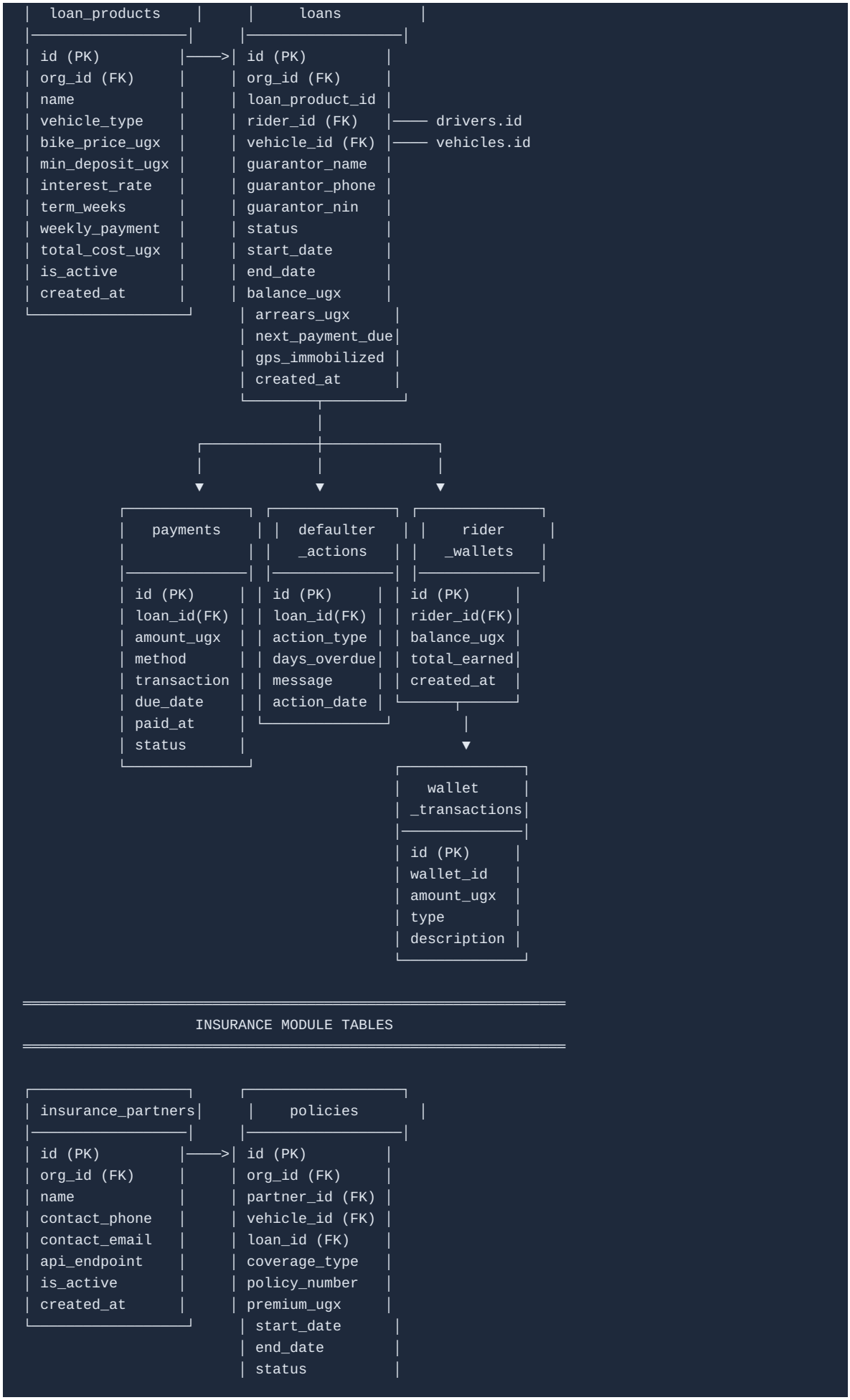
Version: 2.0
Date: June 11, 2026
Database: PostgreSQL 16 + TimescaleDB
ORM: Prisma

---

1. Entity Relationship Diagram


organizations
id (PK)
name
slug
plan
logo_url
address
phone
settings (JSON)
created_at
updated_at
users
id (PK)
org_id (FK)
email
password_hash
role
name
phone
is_active
last_login
created_at
updated_at
drivers
id (PK)
org_id (FK)
name
phone
email
license_number
license_expiry
is_active
created_at
updated_at
vehicles
id (PK)
org_id (FK)
driver_id (FK)
drivers.id
name
plate_number
vin
model
year
type
status
battery_capacity
max_range_km
odometer_km
depot
device_id
is_active
created_at
updated_at
locations
(TimescaleDB)
time
vehicle_id (FK)
battery_logs
(TimescaleDB)
time
vehicle_id (FK)
charging_sessions
(TimescaleDB)
id (PK)
vehicle_id (FK)


lat
lng
speed_kmh
heading
altitude_m
ignition
odometer_km
battery_level_pct
level_pct
soh_pct
voltage_v
temperature_c
charging
current_a
estimated_range
start_time
end_time
start_level_pct
end_level_pct
energy_kwh
charger_id
cost_ugx
created_at
trips
id (PK)
vehicle_id (FK)
driver_id (FK)
start_time
end_time
start_lat
start_lng
end_lat
end_lng
distance_km
avg_speed_kmh
max_speed_kmh
duration_min
created_at
maintenance
id (PK)
vehicle_id (FK)
type
description
status
scheduled_date
completed_date
odometer_km
cost_ugx
notes
created_at
updated_at
alerts
id (PK)
org_id (FK)
vehicle_id (FK)
type
severity
message
data (JSON)
is_acknowledged
acknowledged_by
acknowledged_at
created_at
alert_settings
id (PK)
org_id (FK)
alert_type
threshold
enabled
notify_email
notify_sms
notify_inapp
created_at
updated_at
telemetry_keys
id (PK)
org_id (FK)
name
api_key_hash
is_active
last_used_at
created_at
FINANCING MODULE TABLES


loan_products
id (PK)
org_id (FK)
name
vehicle_type
bike_price_ugx
min_deposit_ugx
interest_rate
term_weeks
weekly_payment
total_cost_ugx
is_active
created_at
loans
id (PK)
org_id (FK)
loan_product_id
rider_id (FK)
drivers.id
vehicle_id (FK)
vehicles.id
guarantor_name
guarantor_phone
guarantor_nin
status
start_date
end_date
balance_ugx
arrears_ugx
next_payment_due
gps_immobilized
created_at
payments
id (PK)
loan_id(FK)
amount_ugx
method
transaction
due_date
paid_at
status
defaulter_actions
id (PK)
loan_id(FK)
action_type
days_overdue
message
action_date
rider_wallets
id (PK)
rider_id(FK)
balance_ugx
total_earned
created_at
wallet_transactions
id (PK)
wallet_id
amount_ugx
type
description
INSURANCE MODULE TABLES
insurance_partners
id (PK)
org_id (FK)
name
contact_phone
contact_email
api_endpoint
is_active
created_at
policies
id (PK)
org_id (FK)
partner_id (FK)
vehicle_id (FK)
loan_id (FK)
coverage_type
policy_number
premium_ugx
start_date
end_date
status


document_url
created_at
claims
id (PK)
policy_id (FK)
incident_date
description
amount_claimed
status
resolution
created_at
BANKING MODULE TABLES
bank_partners
id (PK)
org_id (FK)
name
contact_email
api_endpoint
is_active
created_at
mobile_money _statements
id (PK)
rider_id (FK)
provider
phone_number
statement_period
raw_data (JSON)
created_at
income_verifications
id (PK)
rider_id (FK)
statement_id (FK)
avg_daily_income
income_score
consistency_pct
trend
created_at
income _certificates
id (PK)
verification_id
rider_id (FK)
bank_partner_id
certificate_url
income_score
valid_until
created_at

2. Prisma Schema (v2.0 — Complete)

```
---

## 2. Prisma Schema

```prisma
// fleetspark/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ─── Organization ──────────────────────────────────────

model Organization {
  id          String   @id @default(uuid())
  name        String
  slug        String   @unique
  plan        Plan     @default(FREE)
  logoUrl     String?
  address     String?
  phone       String?
  settings    Json     @default("{}")
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  // Relations
  users           User[]
  drivers         Driver[]
  vehicles        Vehicle[]
  alerts          Alert[]
  alertSettings   AlertSetting[]
  telemetryKeys   TelemetryKey[]

  @@map("organizations")
}

enum Plan {
  FREE
  STARTER
  GROWTH
  ENTERPRISE
}

// ─── User ──────────────────────────────────────────────

model User {
  id            String       @id @default(uuid())
  orgId         String       @map("org_id")
  email         String       @unique
  passwordHash  String       @map("password_hash")
  role          UserRole     @default(VIEWER")
  name          String
  phone         String?
  isActive      Boolean      @default(true) @map("is_active")
  lastLogin     DateTime?    @map("last_login")
  createdAt     DateTime     @default(now()) @map("created_at")
  updatedAt     DateTime     @updatedAt @map("updated_at")
```
```

```
  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  acknowledgedAlerts Alert[] @relation("AlertAcknowledger")

  @@map("users")
}

enum UserRole {
  ADMIN
  FLEET_MANAGER
  DRIVER
  VIEWER
}

// ─── Driver ──────────────────────────────────────────────

model Driver {
  id             String       @id @default(uuid())
  orgId          String       @map("org_id")
  name           String
  phone          String
  email          String?
  licenseNumber  String?      @map("license_number")
  licenseExpiry  DateTime?    @map("license_expiry")
  isActive       Boolean      @default(true) @map("is_active")
  createdAt      DateTime     @default(now()) @map("created_at")
  updatedAt      DateTime     @updatedAt @map("updated_at")

  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  vehicles      Vehicle[]
  trips         Trip[]

  @@unique([orgId, licenseNumber])
  @@map("drivers")
}

// ─── Vehicle ─────────────────────────────────────────────

model Vehicle {
  id               String       @id @default(uuid())
  orgId            String       @map("org_id")
  driverId         String?      @map("driver_id")
  name             String
  plateNumber      String       @map("plate_number")
  vin              String?      @unique
  model            String
  year             Int
  type             VehicleType  @default(BUS)
  status           VehicleStatus @default(IDLE)
  batteryCapacity  Float?       @map("battery_capacity") // kWh
  maxRangeKm       Int?         @map("max_range_km")
  odometerKm       Float        @default(0) @map("odometer_km")
  depot            String?
  deviceId         String?      @map("device_id")
  isActive         Boolean      @default(true) @map("is_active")
  createdAt        DateTime     @default(now()) @map("created_at")
  updatedAt        DateTime     @updatedAt @map("updated_at")

  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  driver        Driver?      @relation(fields: [driverId], references: [id], onDelete: SetNull)
  locations     Location[]
```

```
  batteryLogs   BatteryLog[]
  chargingSessions ChargingSession[]
  trips         Trip[]
  maintenance   Maintenance[]
  alerts        Alert[]

  @@unique([orgId, plateNumber])
  @@index([orgId, status])
  @@map("vehicles")
}

enum VehicleType {
  BUS
  COACH
  CAR
  VAN
  TRUCK
}

enum VehicleStatus {
  IN_TRANSIT
  CHARGING
  IDLE
  MAINTENANCE
  OFFLINE
}

// ─── Location (TimescaleDB hypertable) ──────────────────────

model Location {
  time            DateTime     @map("time")
  vehicleId       String       @map("vehicle_id")
  lat             Float
  lng             Float
  speedKmh        Float?       @map("speed_kmh")
  heading         Float?
  altitudeM       Float?       @map("altitude_m")
  ignition        Boolean      @default(false)
  odometerKm      Float?       @map("odometer_km")
  batteryLevelPct Int?         @map("battery_level_pct")

  // Relations
  vehicle  Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)

  @@index([vehicleId, time(sort: Desc)])
  @@map("locations")
}

// ─── Battery Log (TimescaleDB hypertable) ──────────────────

model BatteryLog {
  time            DateTime     @map("time")
  vehicleId       String       @map("vehicle_id")
  levelPct        Int          @map("level_pct")
  sohPct          Int?         @map("soh_pct")
  voltageV        Float?       @map("voltage_v")
  temperatureC    Float?       @map("temperature_c")
  charging        Boolean      @default(false)
  currentA        Float?       @map("current_a")
  estimatedRangeKm Int?        @map("estimated_range_km")

  // Relations
  vehicle  Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)
```

```
  @@index([vehicleId, time(sort: Desc)])
  @@map("battery_logs")
}

// ─── Charging Session (TimescaleDB hypertable) ──────────────

model ChargingSession {
  id              String       @id @default(uuid())
  vehicleId       String       @map("vehicle_id")
  startTime       DateTime     @map("start_time")
  endTime         DateTime?    @map("end_time")
  startLevelPct   Int          @map("start_level_pct")
  endLevelPct     Int?         @map("end_level_pct")
  energyKwh       Float?       @map("energy_kwh")
  chargerId       String?      @map("charger_id")
  costUgx         Int?         @map("cost_ugx")
  createdAt       DateTime     @default(now()) @map("created_at")

  // Relations
  vehicle  Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)

  @@index([vehicleId, startTime(sort: Desc)])
  @@map("charging_sessions")
}

// ─── Trip ────────────────────────────────────────────────────

model Trip {
  id            String       @id @default(uuid())
  vehicleId     String       @map("vehicle_id")
  driverId      String?      @map("driver_id")
  startTime     DateTime     @map("start_time")
  endTime       DateTime?    @map("end_time")
  startLat      Float?       @map("start_lat")
  startLng      Float?       @map("start_lng")
  endLat        Float?       @map("end_lat")
  endLng        Float?       @map("end_lng")
  distanceKm    Float?       @map("distance_km")
  avgSpeedKmh   Float?       @map("avg_speed_kmh")
  maxSpeedKmh   Float?       @map("max_speed_kmh")
  durationMin   Int?         @map("duration_min")
  createdAt     DateTime     @default(now()) @map("created_at")

  // Relations
  vehicle  Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)
  driver   Driver?  @relation(fields: [driverId], references: [id], onDelete: SetNull)

  @@index([vehicleId, startTime(sort: Desc)])
  @@map("trips")
}

// ─── Maintenance ─────────────────────────────────────────────

model Maintenance {
  id              String              @id @default(uuid())
  vehicleId       String              @map("vehicle_id")
  type            MaintenanceType
  description     String?
  status          MaintenanceStatus   @default(SCHEDULED)
  scheduledDate   DateTime            @map("scheduled_date")
  completedDate   DateTime?           @map("completed_date")
  odometerKm      Float?              @map("odometer_km")
  costUgx         Int?                @map("cost_ugx")
  notes           String?
```

```
  createdAt       DateTime           @default(now()) @map("created_at")
  updatedAt       DateTime           @updatedAt @map("updated_at")

  // Relations
  vehicle  Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)

  @@index([vehicleId, status])
  @@index([scheduledDate])
  @@map("maintenance")
}

enum MaintenanceType {
  ROUTINE
  REPAIR
  INSPECTION
  BATTERY_SERVICE
  TIRE
  BRAKE
  OTHER
}

enum MaintenanceStatus {
  SCHEDULED
  COMPLETED
  CANCELLED
}

// ─── Alert ──────────────────────────────────────────────

model Alert {
  id              String       @id @default(uuid())
  orgId           String       @map("org_id")
  vehicleId       String       @map("vehicle_id")
  type            AlertType
  severity        AlertSeverity @default(WARNING)
  message         String
  data            Json?        // Additional context data
  isAcknowledged  Boolean      @default(false) @map("is_acknowledged")
  acknowledgedBy  String?      @map("acknowledged_by")
  acknowledgedAt  DateTime?    @map("acknowledged_at")
  createdAt       DateTime     @default(now()) @map("created_at")

  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  vehicle       Vehicle      @relation(fields: [vehicleId], references: [id], onDelete: Cascade)
  acknowledger  User?        @relation("AlertAcknowledger", fields: [acknowledgedBy], references: [id], onDelete:

  @@index([orgId, createdAt(sort: Desc)])
  @@index([vehicleId, createdAt(sort: Desc)])
  @@index([isAcknowledged])
  @@map("alerts")
}

enum AlertType {
  LOW_BATTERY
  OFFLINE
  MAINTENANCE_DUE
  GEOFENCE_EXIT
  HIGH_TEMPERATURE
  RAPID_BATTERY_DROP
  CHARGING_COMPLETED
  VEHICLE_IDLE_TOO_LONG
}
```

```
enum AlertSeverity {
  INFO
  WARNING
  CRITICAL
}

// ─── Alert Settings ──────────────────────────────────────

model AlertSetting {
  id            String       @id @default(uuid())
  orgId         String       @map("org_id")
  alertType     AlertType    @map("alert_type")
  threshold     Json?        // e.g., {"level_pct": 20} for LOW_BATTERY
  enabled       Boolean      @default(true)
  notifyEmail   Boolean      @default(true) @map("notify_email")
  notifySms     Boolean      @default(false) @map("notify_sms")
  notifyInApp   Boolean      @default(true) @map("notify_in_app")
  createdAt     DateTime     @default(now()) @map("created_at")
  updatedAt     DateTime     @updatedAt @map("updated_at")

  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)

  @@unique([orgId, alertType])
  @@map("alert_settings")
}

// ─── Telemetry API Keys ──────────────────────────────────

model TelemetryKey {
  id            String       @id @default(uuid())
  orgId         String       @map("org_id")
  name          String
  apiKeyHash    String       @map("api_key_hash")
  isActive      Boolean      @default(true) @map("is_active")
  lastUsedAt    DateTime?    @map("last_used_at")
  createdAt     DateTime     @default(now()) @map("created_at")

  // Relations
  organization  Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)

  @@map("telemetry_keys")
}
```

3. TimescaleDB Setup

After Prisma migration, run these SQL commands to convert tables to hypertables:

```
-- Convert locations to hypertable (partitioned by time)
SELECT create_hypertable('locations', 'time',
  chunk_time_interval => INTERVAL '1 day',
  if_not_exists => TRUE
);

-- Convert battery_logs to hypertable
SELECT create_hypertable('battery_logs', 'time',
  chunk_time_interval => INTERVAL '1 day',
  if_not_exists => TRUE
);

-- Convert charging_sessions to hypertable
SELECT create_hypertable('charging_sessions', 'start_time',
  chunk_time_interval => INTERVAL '7 days',
  if_not_exists => TRUE
);

-- Set retention policies (auto-delete old data)
SELECT add_retention_policy('locations', INTERVAL '90 days');
SELECT add_retention_policy('battery_logs', INTERVAL '180 days');
SELECT add_retention_policy('charging_sessions', INTERVAL '365 days');

-- Create continuous aggregates for fast dashboard queries
CREATE MATERIALIZED VIEW locations_hourly
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 hour', time) AS bucket,
  vehicle_id,
  AVG(speed_kmh) as avg_speed,
  MAX(speed_kmh) as max_speed,
  AVG(battery_level_pct) as avg_battery,
  MIN(battery_level_pct) as min_battery,
  last(lat, time) as last_lat,
  last(lng, time) as last_lng
FROM locations
GROUP BY bucket, vehicle_id;

CREATE MATERIALIZED VIEW battery_daily
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 day', time) AS bucket,
  vehicle_id,
  AVG(level_pct) as avg_level,
  MIN(level_pct) as min_level,
  MAX(level_pct) as max_level,
  AVG(soh_pct) as avg_soh,
  AVG(temperature_c) as avg_temp,
  COUNT(*) FILTER (WHERE charging) as charging_minutes
FROM battery_logs
GROUP BY bucket, vehicle_id;
```

4. Indexes Summary

Table	Index	Purpose
users	email (unique)	Login lookup
vehicles	org_id + plate_number (unique)	Prevent duplicate plates per org
vehicles	org_id + status	Dashboard filtering
locations	vehicle_id + time (desc)	Vehicle location history
battery_logs	vehicle_id + time (desc)	Battery history queries
charging_sessions	vehicle_id + start_time (desc)	Charging history
trips	vehicle_id + start_time (desc)	Trip history
maintenance	vehicle_id + status	Active maintenance lookup
maintenance	scheduled_date	Upcoming maintenance query
alerts	org_id + created_at (desc)	Alert list
alerts	vehicle_id + created_at (desc)	Vehicle alerts
alerts	is_acknowledged	Unacknowledged alerts filter
alert_settings	org_id + alert_type (unique)	Settings lookup

5. Seed Data

```
// prisma/seed.ts

// Demo Organization
const org = await prisma.organization.create({
  data: {
    name: 'Kiira Motors Demo',
    slug: 'kiira-demo',
    plan: 'ENTERPRISE',
    settings: {
      defaultDieselPrice: 5500,
      defaultEmissionFactor: 2.68,
      timezone: 'Africa/Kampala',
    },
  },
});

// Admin User (password: "demo123")
const admin = await prisma.user.create({
  data: {
    orgId: org.id,
    email: 'admin@fleetspark.demo',
    passwordHash: '$2b$10$...', // bcrypt hash of "demo123"
    role: 'ADMIN',
    name: 'Fleet Admin',
    phone: '+256772000000',
  },
});

// Demo Vehicles (5 buses)
const vehicles = await Promise.all([
  { name: 'Kayoola-001', plate: 'UAX 123A', model: 'Kayoola EVS', type: 'BUS', batteryCapacity: 250, maxRangeKm:
  { name: 'Kayoola-002', plate: 'UAX 124A', model: 'Kayoola EVS', type: 'BUS', batteryCapacity: 250, maxRangeKm:
  { name: 'Kayoola-003', plate: 'UAX 125A', model: 'Kayoola EVS', type: 'BUS', batteryCapacity: 250, maxRangeKm:
  { name: 'Kayoola-004', plate: 'UAX 126A', model: 'Kayoola EVS', type: 'BUS', batteryCapacity: 250, maxRangeKm:
  { name: 'Kayoola-005', plate: 'UAX 127A', model: 'Kayoola EVS', type: 'BUS', batteryCapacity: 250, maxRangeKm:
].map(v => prisma.vehicle.create({
  data: { ...v, orgId: org.id, year: 2025, status: 'IDLE', odometerKm: 0 },
})));

// Demo Drivers
const drivers = await Promise.all([
  { name: 'John Kato', phone: '+256772111111', licenseNumber: 'DL001' },
  { name: 'Sarah Nakiwala', phone: '+256772222222', licenseNumber: 'DL002' },
  { name: 'David Okello', phone: '+256772333333', licenseNumber: 'DL003' },
].map(d => prisma.driver.create({ data: { ...d, orgId: org.id } })));

// Assign drivers to vehicles
for (let i = 0; i < Math.min(vehicles.length, drivers.length); i++) {
  await prisma.vehicle.update({
    where: { id: vehicles[i].id },
    data: { driverId: drivers[i].id },
  });
}

// Default Alert Settings
const alertTypes = ['LOW_BATTERY', 'OFFLINE', 'MAINTENANCE_DUE', 'GEOFENCE_EXIT', 'HIGH_TEMPERATURE', 'RAPID_BATT
for (const type of alertTypes) {
  await prisma.alertSetting.create({
    data: {
      orgId: org.id,
      alertType: type as any,
      threshold: type === 'LOW_BATTERY' ? { level_pct: 20 } :
                 type === 'OFFLINE' ? { minutes: 30 } :
```

```
                type === 'HIGH_TEMPERATURE' ? { temp_c: 45 } : {},
      enabled: true,
      notifyEmail: true,
      notifySms: false,
      notifyInApp: true,
    },
  });
}

// Telemetry API Key
await prisma.telemetryKey.create({
  data: {
    orgId: org.id,
    name: 'Demo GPS Device',
    apiKeyHash: '$2b$10$...', // bcrypt hash of demo key
  },
});
```

---

6. Migration Commands

```
# Initial setup
npx prisma migrate dev --name init

# After schema changes
npx prisma migrate dev --name <description>

# Generate client (after any schema change)
npx prisma generate

# Seed database
npx prisma db seed

# Reset database (dev only)
npx prisma migrate reset

# Open Prisma Studio (GUI)
npx prisma studio
```

---

7. Data Retention Policy

Data Type	Retention	Action
Locations	90 days	Auto-delete via TimescaleDB policy
Battery logs	180 days	Auto-delete via TimescaleDB policy
Charging sessions	365 days	Auto-delete via TimescaleDB policy
Trips	Forever	No retention (aggregated data)
Maintenance	Forever	No retention
Alerts	1 year	Manual archive job
Audit logs	1 year	Manual archive job

8. Key Queries

Fleet Dashboard Summary

```
SELECT
  COUNT(*) as total_vehicles,
  COUNT(*) FILTER (WHERE status = 'IN_TRANSIT') as active_now,
  COUNT(*) FILTER (WHERE status = 'CHARGING') as charging_now,
  COUNT(*) FILTER (WHERE status = 'MAINTENANCE') as in_maintenance,
  COUNT(*) FILTER (WHERE status = 'OFFLINE') as offline
FROM vehicles
WHERE org_id = $1 AND is_active = true;
```

Latest Location Per Vehicle

```
SELECT DISTINCT ON (vehicle_id)
  vehicle_id, lat, lng, speed_kmh, battery_level_pct, time
FROM locations
WHERE vehicle_id IN (SELECT id FROM vehicles WHERE org_id = $1)
ORDER BY vehicle_id, time DESC;
```

Battery History (Last 24h)

```
SELECT time, level_pct, soh_pct, charging, estimated_range_km
FROM battery_logs
WHERE vehicle_id = $1 AND time > NOW() - INTERVAL '24 hours'
ORDER BY time ASC;
```

Unacknowledged Alerts

```
SELECT a.*, v.name as vehicle_name, v.plate_number
FROM alerts a
JOIN vehicles v ON a.vehicle_id = v.id
WHERE a.org_id = $1 AND a.is_acknowledged = false
ORDER BY a.created_at DESC;
```

Daily Distance Per Vehicle

```
SELECT
  vehicle_id,
  date_trunc('day', start_time) as day,
  SUM(distance_km) as total_km,
  COUNT(*) as trip_count
FROM trips
WHERE org_id = $1 AND start_time > NOW() - INTERVAL '30 days'
GROUP BY vehicle_id, date_trunc('day', start_time)
ORDER BY day DESC;
```